LAB 7

# Fuzzing for Input Validation Bugs

Marathon Registration


| NAME | SRN | SECTION |
|---|---|---|
| Chaithanya B | PES1UG24CS123 | B |


## Objective

Use property-based fuzz testing with Hypothesis to find input validation bugs in the marathon registration parser, then harden the code so that all three functions handle any input without crashing.

## Results

| BUGGY MODULE | FIXED MODULE | FUZZ TESTS |
|---|---|---|
| 3 / 3 failed | 3 / 3 passed | 3 properties |

## Approach

One Hypothesis test was written for each function (`format_bib_name`, `parse_finish_time`, `average_split`). Each feeds `st.text() | st.none()` into the function and asserts that it never raises an exception. The tests failed on the buggy module, the module was then hardened with type checks and safe parsing, and the same tests passed on the fixed version.

Screenshots SS1 to SS4, the bug analysis, reflection answers and complete code follow. A passing no-crash property shows the functions are robust to arbitrary input; it does not by itself prove that every output is semantically correct.

# Buggy module and failing fuzz tests

**STEP 2 & 4 / SS1 AND SS2**

## SS1: buggy module run

```
(.venv) PS D:\Documents\Sem-5\SE\SE-Labs-PES1UG24CS123\L6_L7_60_marathon_registration\Lab7> python marathon_regi
stration_parser.py
==== Running format_bib_name ====
Enter the name to print on the bib: Chaithanya
Bib name: CHAITHANYA

==== Running parse_finish_time ====
Enter a finish time as H:MM:SS (e.g. 3:45:10): 4:00:10
Finish time in seconds: 14410

==== Running average_split ====
Enter your 5 km split times in minutes separated by spaces (e.g. 25 26 24): 30 33 35
Average split (minutes): 32.666666666666664
(.venv) PS D:\Documents\Sem-5\SE\SE-Labs-PES1UG24CS123\L6_L7_60_marathon_registration\Lab7>
```

With well-formed input the buggy module works normally, which is why these bugs go unnoticed in manual testing.

## SS2: Hypothesis tests failing on the buggy module

```
Problems 1  Output  Debug Console  Terminal  Ports   powershell - Lab7

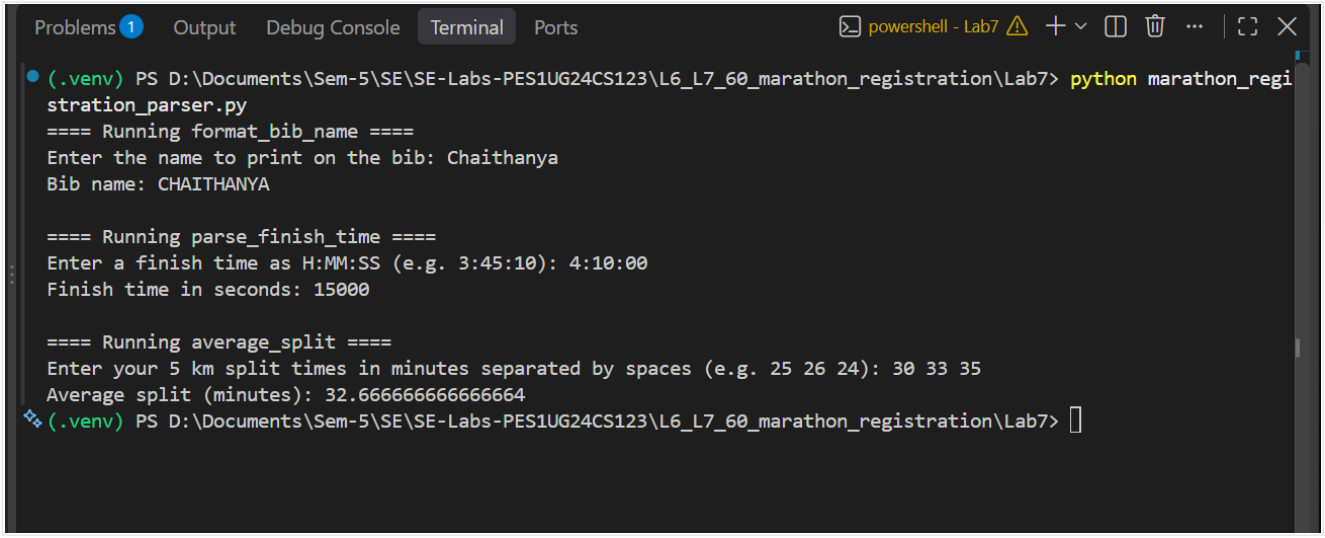
(.venv) PS D:\Documents\Sem-5\SE\SE-Labs-PES1UG24CS123\L6_L7_60_marathon_registration\Lab7> python -m pytest tes
t_marathon_registration_parser.py --tb=short
    |     splits = [int(s) for s in text.split()]
    |                               ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\Documents\Sem-5\SE\SE-Labs-PES1UG24CS123\L6_L7_60_marathon_registration\Lab7\test_marathon_regi
stration_parser.py", line 23, in test_average_split_no_crash
    |     assert False, f"average_split crashed with input {repr(s)}: {e}"
    | AssertionError: average_split crashed with input None: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_average_split_no_crash(
    |     s=None,
    | )
    +------------------------------------
========================================== short test summary info ==========================================
FAILED test_marathon_registration_parser.py::test_format_bib_name_no_crash - AssertionError: format_bib_name cra
shed with input None: 'NoneType' object has no attribute 'strip'
FAILED test_marathon_registration_parser.py::test_parse_finish_time_no_crash - ExceptionGroup: Hypothesis found
2 distinct failures. (2 sub-exceptions)
FAILED test_marathon_registration_parser.py::test_average_split_no_crash - ExceptionGroup: Hypothesis found 3 di
stinct failures. (3 sub-exceptions)
========================================== 3 failed in 3.02s ==========================================
(.venv) PS D:\Documents\Sem-5\SE\SE-Labs-PES1UG24CS123\L6_L7_60_marathon_registration\Lab7>
```

All 3 tests failed. Hypothesis reported a minimal failing input for each function, for example `None` for `format_bib_name` (`'NoneType' object has no attribute 'strip'`) and for `average_split`.

# Fixed module and passing fuzz tests

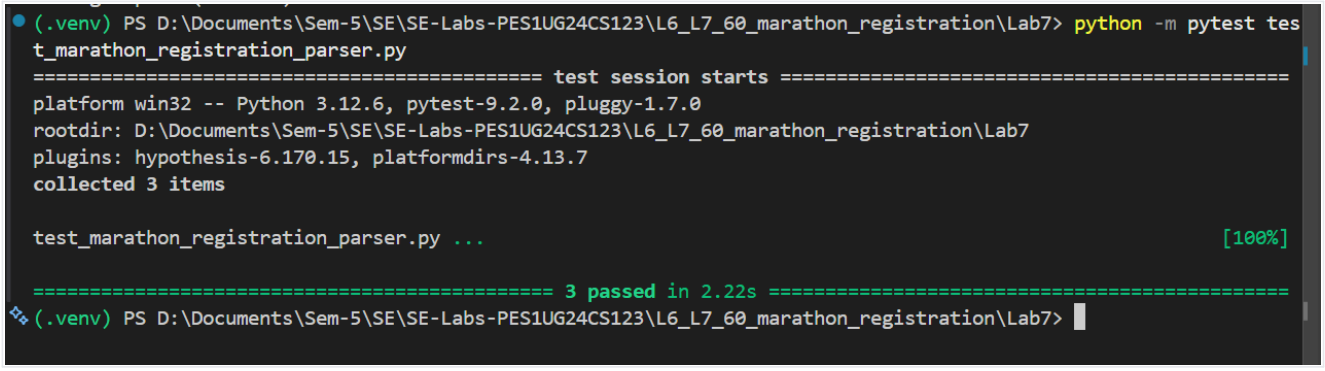
**STEP 5 & 6 / SS3 AND SS4**

## SS3: fixed module run




Valid input still produces correct results (`4:10:00` gives 15000 seconds; the average of 30, 33 and 35 is 32.67 minutes).

## SS4: Hypothesis tests passing on the fixed module




All 3 tests passed against the fixed module, so none of the generated inputs crashed any function.

# Bugs found and fixes

**ANALYSIS**

| FUNCTION | BUG FOUND BY FUZZING | FIX APPLIED |
|---|---|---|
| **format_bib_name** | `None` (or any non-string) crashes on `.strip()` with AttributeError. Empty or whitespace-only names gave a blank bib. | Return `"N/A"` if the value is not a string or is empty after stripping; otherwise strip, upper-case and cut to 12 characters. |
| **parse_finish_time** | `None` crashes on `.split()`. Malformed text (wrong number of parts, non-numeric parts) raised an unhandled exception. | Type check, require exactly 3 parts, wrap `int()` in try/except ValueError, reject negatives and return 0 for any invalid value. |
| **average_split** | `None` crashes on `.split()`. A non-numeric token made `int()` raise ValueError. Empty or blank text produced ZeroDivisionError. | Type check, parse each token with `float()` inside try/except and skip bad tokens, and return 0.0 when no valid splits remain. |

## Reflection

**1. What kinds of bugs did fuzzing uncover, and why were they missed by the normal run?**

Fuzzing exposed unhandled `None` values, malformed or non-numeric text, and empty or blank strings. The code assumed every input was well formed, so the normal run with sensible values (SS1) never touched these paths. Hypothesis generated such inputs automatically and shrank each failure to a minimal example.

**2. How did you make the functions robust?**

Each function now validates the input type first, parses defensively with try/except, and returns a safe default (`"N/A"`, `0` or `0.0`) instead of raising. Behaviour for valid input is unchanged, as SS3 shows.

**3. What are the strengths and limits of this fuzzing approach?**

It finds edge cases a person would not think to write and replays failing inputs for debugging. However, the property only checks that the code does not crash, not that results are correct; for instance, `average_split` still accepts tokens such as `nan` or `inf`. Combining fuzz tests with value-checking unit tests would give stronger guarantees.

# Code

## marathon_registration_parser.py (fixed)

```
def format_bib_name(name):
    """
    Upper-cases a runner's name and cuts it to 12 characters for the bib.
    Safely handles None, non-string, empty, or whitespace-only inputs.
    """
    if not isinstance(name, str) or not name.strip():
        return "N/A"
    return name.strip().upper()[:12]

def parse_finish_time(text):
    """
    Converts a finish time written as H:MM:SS into seconds.
    Returns 0 if format is invalid or missing.
    """
    if not isinstance(text, str):
        return 0
    parts = text.strip().split(':')
    if len(parts) != 3:
        return 0
    try:
        hours, minutes, seconds = int(parts[0]), int(parts[1]), int(parts[2])
        if hours < 0 or minutes < 0 or seconds < 0:
            return 0
        return hours * 3600 + minutes * 60 + seconds
    except ValueError:
        return 0

def average_split(text):
    """
    Averages split times given in minutes, separated by spaces.
    Returns 0.0 if inputs are invalid or no valid splits exist.
    """
    if not isinstance(text, str):
        return 0.0
    splits = []
    for token in text.strip().split():
        try:
            splits.append(float(token))
        except ValueError:
            continue
    if not splits:
        return 0.0
    return sum(splits) / len(splits)

def main():
    print("==== Running format_bib_name ====")
    value = input("Enter the name to print on the bib: ")
    print("Bib name:", format_bib_name(value))

    print("\n==== Running parse_finish_time ====")
    value = input("Enter a finish time as H:MM:SS (e.g. 3:45:10): ")
    print("Finish time in seconds:", parse_finish_time(value))

    print("\n==== Running average_split ====")
    value = input("Enter your 5 km split times in minutes separated by spaces (e.g. 25 26 24): ")
    print("Average split (minutes):", average_split(value))

if __name__ == "__main__":
    main()
```

# Code (continued)

## test_marathon_registration_parser.py (completed Hypothesis tests)

```
from hypothesis import given, strategies as st
from marathon_registration_parser import format_bib_name, parse_finish_time, average_split

@given(st.text() | st.none())
def test_format_bib_name_no_crash(s):
    try:
        format_bib_name(s)
    except Exception as e:
        assert False, f"format_bib_name crashed with input {repr(s)}: {e}"

@given(st.text() | st.none())
def test_parse_finish_time_no_crash(s):
    try:
        parse_finish_time(s)
    except Exception as e:
        assert False, f"parse_finish_time crashed with input {repr(s)}: {e}"

@given(st.text() | st.none())
def test_average_split_no_crash(s):
    try:
        average_split(s)
    except Exception as e:
        assert False, f"average_split crashed with input {repr(s)}: {e}"
```